PRASAD V POTLURI SIDDHARTHA INSTITUTE OF TECHNOLOGY DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

ACADEMIC YEAR: 2026-27

FULL-STACK SOFTWARE ENGINEERING PROJECT REPORT

SHELFSENSE: SMART INVENTORY AND EXPIRY MANAGEMENT PLATFORM USING FIRST EXPIRY FIRST OUT (FEFO) BATCH ALLOCATION

Faculty Name: Ms. Y. SUREKHA

Team Members:

1. Name Roll No
2.

ABSTRACT

Perishable stock mismanagement and product expiration represent significant financial losses, food/medicine waste, and compliance liabilities for retail businesses, supermarkets, pharmacies, and warehouse distributors. Traditional inventory software frequently treats stock as homogenous numeric counters without lot manufacturing timestamps, expiration deadlines, and supplier batch traceability. Consequently, older batches remain hidden on rear shelving until spoiled while newly replenished stock is prematurely sold, resulting in substantial fiscal deficits.

This project presents ShelfSense, an intelligent, multi-tenant enterprise Inventory and Expiry Management SaaS platform built on the MERN stack (MongoDB Atlas, Express.js, React 18, and Node.js) with end-to-end strict TypeScript. ShelfSense implements an automated First Expiry, First Out (FEFO) batch allocation engine that dynamically routes customer point-of-sale checkout deductions to the earliest-expiring available inventory lots. The system incorporates barcode-based product indexing, multi-tier expiry horizon tracking (Expired, <=7 days, <=30 days, <=60 days), vendor procurement order lifecycles with single-click batch generation, an immutable inventory movement ledger, audit logging, and role-based access control (Owner/Admin, Inventory Manager, Cashier). Automated integration tests validate cryptographic authentication, atomic stock consistency under concurrency, and strict multi-tenant boundary segregation with a 100% test pass rate.

KEYWORDS: Inventory Management, Expiry Tracking, First Expiry First Out (FEFO), Multi-Tenant SaaS, MERN Stack, TypeScript, Role-Based Access Control, POS System, MongoDB Atlas.

INTRODUCTION

Retail trade and inventory management form the economic backbone of local commerce, supplying daily essentials, foods, and life-saving pharmaceuticals to communities. As commercial supply chains become increasingly dynamic, managing stock turnover effectively determines business survival. A recurring challenge faced by grocery stores, pharmacies, supermarket chains, and FMCG distributors is managing goods with finite shelf lives.

Perishable goods such as dairy products, baked items, fresh packaged foods, cold beverages, and medical supplies possess strict expiration windows beyond which they cannot be sold. When retail stores lack granular lot-level visibility, expiring products frequently languish undetected on storage shelves while newly replenished stock is accidentally sold first. A customer who purchases an expired item faces health risks, while the retailer suffers reputational harm and financial loss.

Traditional inventory software commonly relies on simple aggregate counters, manual periodic stock audits, or basic point-of-sale registers. Although these tools assist with daily transaction logging, they fail to track the varying expiration dates of multiple concurrent batches for the same product SKU.

Modern web technologies and domain-driven design provide another approach to inventory control by building date-aware intelligence directly into the sales and procurement pipeline. In this project, a full-stack MERN platform developed with strict TypeScript is used to manage batch-level inventory. Instead of depending entirely on manual shelf inspections, the system enforces automated First Expiry, First Out (FEFO) lot allocation at the moment of checkout.

The system first captures product catalog information, registers incoming batches with explicit manufacturing and expiry dates, updates multi-tier expiry status indicators, and monitors stock health. During transactions, it applies FEFO logic to allocate stock from the earliest expiring lots while excluding expired batches from sales.

Furthermore, the proposed system focuses on making enterprise-grade inventory management accessible to small and medium business owners without requiring expensive proprietary hardware. By organizing inventory data into an immutable movement ledger and providing clear dashboard metrics, the platform empowers retailers to make timely discounting decisions, minimize spoilage waste, and optimize working capital.

PROBLEM STATEMENT

Perishable inventory management in retail businesses is susceptible to spoilage losses because conventional software packages do not differentiate between distinct product batches bearing varying expiration dates. Tracking product shelf life manually is labor-intensive, error-prone, and incapable of preventing expired items from being mistakenly scanned at checkout counters.

Therefore, there is an imperative need for an automated, multi-tenant inventory operating system that tracks product lots individually, alerts staff to impending expirations, and

automatically deducts stock using First Expiry, First Out (FEFO) prioritization during customer sales. The proposed project addresses this problem by developing ShelfSense, a production-oriented MERN stack web application with strict TypeScript checking, role-based access control, and complete cloud database isolation.

Objectives of the Project

The main objectives of this project are:

To develop an automated, production-oriented cloud inventory and expiry management platform.

To implement an automated First Expiry, First Out (FEFO) batch allocation engine for retail sales.

To support multiple concurrent batches per SKU with distinct manufacturing and expiration dates.

To classify batches into proactive temporal expiry horizons: Expired, <= 7 days, <= 30 days, and <= 60 days.

To exclude expired product batches automatically from customer sales at checkout.

To build a high-speed Point of Sale (POS) terminal with barcode scanner search and printable receipts.

To engineer an immutable inventory movement ledger capturing sales, receipts, damages, and audits.

To establish vendor procurement workflows with Purchase Orders that instantiate inventory batches upon receipt.

To enforce strict Role-Based Access Control (RBAC) separating Admin, Inventory Manager, and Cashier permissions.

LITERATURE REVIEW

Inventory management and perishability control have been studied across operations research, supply chain engineering, and enterprise software design.

First-In, First-Out (FIFO) vs. First-Expiry, First-Out (FEFO): Traditional warehouse management systems often deploy FIFO, dispatching goods based on receipt date. Research demonstrates that for perishable commodities, FIFO fails whenever supplier delivery delays cause newly received lots to expire earlier than existing stock. FEFO guarantees that goods nearest to expiration are consumed first, significantly reducing waste.

Batch Traceability in Retail: Studies in retail pharmaceutical and grocery logistics emphasize that scalar stock counters fail to support regulatory compliance or targeted product recalls. Granular batch records containing lot identifiers, manufacturing dates, and supplier references provide the traceability necessary for audits and safety governance.

Multi-Tenant Cloud Architectures: Modern SaaS architectures utilize tenant-isolated data models where operational records are partitioned via tenant identifiers. This approach ensures enterprise data segregation and security without requiring the substantial operational cost of separate physical database instances.

Concurrency Control in Retail Transactions: Real-time inventory deductions during simultaneous POS checkouts require atomic updates. When multiple registers access the same batch, database-level atomic conditional increments prevent overselling and negative stock drift.

Based on these paradigms, this project implements a full-stack cloud inventory platform integrating FEFO batch logic, immutable movement ledgers, and multi-tenant MongoDB Atlas clustering.

The architecture incorporates secure session handling via JWT and refresh token rotation, strict schema validation using Zod, and a responsive frontend interface powered by React and Tailwind CSS.

EXISTING SYSTEM

Existing retail inventory systems deployed by small and medium businesses primarily rely on manual spreadsheets, standalone electronic cash registers, or basic generic inventory software.

Spreadsheet-based tracking depends on periodic manual stock counts, leading to data entry lag, human error, and lack of real-time multi-user checkout synchronization. Standalone electronic cash registers record daily sales totals but lack cloud synchronization, lot-level batch tracking, and automated expiration monitoring.

Generic inventory databases increment and decrement total unit quantities without storing manufacturing or expiration timestamps. Consequently, staff cannot determine which shelf items are approaching expiration without physically picking up and inspecting individual containers.

LIMITATIONS OF THE EXISTING SYSTEM

The following are common limitations of existing approaches:

Inability to track multiple batches with different expiration dates for the same product SKU.

Absence of automated FEFO allocation logic during cashier sales checkout.

Significant financial loss resulting from undiscovered product expiration on retail shelves.

Risk of accidentally selling expired foods or medicines to customers.

Lack of an immutable audit trail documenting inventory damages, adjustments, and customer returns.

Absence of multi-tenant cloud accessibility for multi-store business owners.

Vulnerability to inventory discrepancies during concurrent register operations.

PROPOSED SYSTEM

The proposed ShelfSense platform introduces an intelligent, batch-centric cloud inventory management system designed to eliminate perishable waste and streamline retail workflows. ShelfSense maintains an active registry of all product batches, indexing lot numbers, manufacturing dates, expiration dates, unit purchase costs, and supplier links.

During customer checkout, the platform's FEFO engine dynamically sorts non-expired batches in ascending order of expiration date and iteratively allocates required quantities from the earliest-expiring lots. Expired batches are automatically quarantined, preventing accidental sales.

The platform also provides vendor purchase order lifecycles with single-click batch creation upon goods receipt, immutable movement ledger tracking, role-based access control, and a responsive business analytics dashboard.

Automated integration testing validates cryptographic authentication, concurrent inventory consistency, and strict multi-tenant boundary segregation.

ADVANTAGES OF THE PROPOSED SYSTEM

Automates First Expiry, First Out (FEFO) stock deduction during customer checkouts.
Eliminates retail waste by proactively identifying lots nearing expiration within 7, 30, and 60 days.
Prevents the distribution of expired goods through automated POS checkout quarantine.
Enables rapid product lookup and scanning via SKU and barcode indexing.
Maintains an immutable historical ledger of all stock-in, stock-out, damage, and adjustment movements.
Streamlines supplier procurement with automatic inventory batch creation upon purchase order receipt.
Enforces granular operational role permissions for Business Owners, Inventory Managers, and Cashiers.
Guarantees complete multi-tenant data privacy and isolation on cloud databases.

PROPOSED SYSTEM ARCHITECTURE

SYSTEM REQUIREMENTS

Hardware requirements
Computer, laptop, or tablet terminal
Processor: Intel Core i3 / AMD Ryzen 3 or higher
Memory: 4 GB minimum (8 GB recommended for development environment)
Peripherals: Standard USB/Bluetooth barcode scanner and display screen
Software requirements
Node.js (v18.x or v20.x LTS runtime) and npm package manager
Web Browser: Google Chrome, Mozilla Firefox, or Microsoft Edge
Database: MongoDB Atlas Cloud Cluster or MongoDB Community Server 7.0
Backend: Express.js 4.21, Mongoose 8.9, Zod 3.24, bcryptjs, jsonwebtoken
Frontend: React 18, Vite 6, Tailwind CSS 3, TanStack Query 5, Zustand 5, Recharts 2
Testing: Vitest 2.1 and Supertest 7.0
Security: Helmet HTTP headers, CORS origin whitelisting, rate-limiting
Operating System: Windows 10/11, macOS, or Ubuntu Linux
Data format: RESTful JSON over TLS/HTTPS
Programming language: TypeScript (Strict mode across client and server)
Database driver: Mongoose ODM with embedded memory-server fallback
Cloud persistence: MongoDB Atlas multi-tenant cluster (Cluster0)

DATASET AND DATA COLLECTION DESCRIPTION

This project manages operational business data structured across 11 relational-modeled MongoDB collections linked via ObjectIds and enforced with multi-tenant compound indexes.

The collections capture all commercial transactions, inventory balances, user credentials, and security audit events. Each operational record is bound to a specific business tenant ID. Input data is strictly sanitized and validated using Zod schemas before persistence. The core collections include:

Collection 1: Businesses, Users, and Roles (Admin, Manager, Cashier)

Collection 2: Products, ProductBatches, and InventoryMovements

Collection 3: Suppliers, PurchaseOrders, Sales, Notifications, and AuditLogs

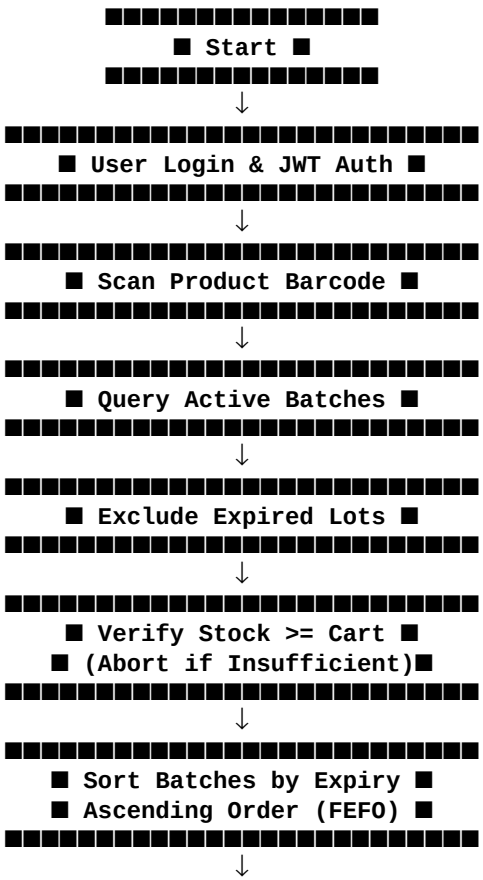
The development database includes seed data featuring 1 retail supermarket, 3 role accounts, 6 product categories, 4 verified suppliers, 22 products, and 44 multi-dated batches.

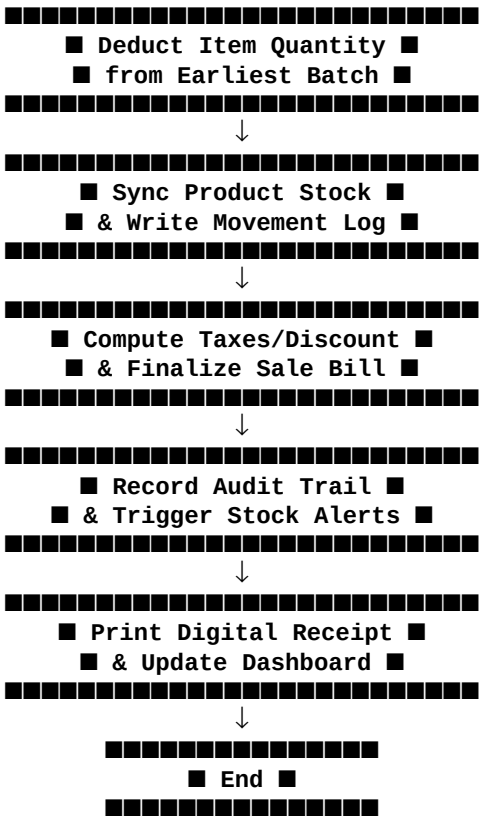
METHODOLOGY

The project follows a sequence of operational steps, from business onboarding to POS checkout and inventory tracking.

1. Onboard business tenant and configure store settings
2. Register role-based user accounts (Admin, Manager, Cashier)
3. Catalog product categories and create SKUs with barcodes
4. Register suppliers and establish procurement payment terms
5. Issue purchase orders and receive delivered goods
6. Instantiate product batches with manufacturing and expiry dates
7. Continuously monitor shelf-life across four expiry horizons
8. Scan barcodes at the POS checkout terminal and apply FEFO logic
9. Deduct stock atomically, log movement ledger, and print receipt

FLOW CHART:





DATA PREPROCESSING AND VALIDATION

Data preprocessing converts raw inventory and transaction payloads into validated, sanitized records that can be stored securely in the database.

First, request bodies are validated using Zod schemas that trim strings, enforce positive pricing decimals, verify valid ObjectId formats, and strip unauthorized parameters.

Next, manufacturing and expiration dates are parsed and normalized to UTC date objects to avoid timezone calculation errors across geographic regions.

Since product lots arrive with varying expiration dates, the system calculates the remaining days of shelf life relative to the server clock.

This process categorizes each batch into its respective expiry horizon and updates active availability flags for point-of-sale operations.

FIRST EXPIRY, FIRST OUT (FEFO) BATCH ALLOCATION

FEFO allocation maps sales requests to specific inventory lots based on expiration chronology rather than general product quantities.

The system does not deduct from a simple numeric balance. Instead, it queries all non-expired batches for the SKU and allocates stock sequentially.

Illustrative example:

Product: Organic Whole Milk 1 Gallon (SKU: DAI-MILK-001) - Requested: 12 units

is allocated across batches as:

Lot A (Expires in 4 days): 10 units allocated -> Lot B (Expires in 14 days): 2 units allocated

These lot deductions are atomic, preventing concurrency conflicts when multiple registers operate simultaneously.

FEFO allocation ensures that items nearest to expiration are cleared first, minimizing store spoilage and preserving profit margins.

MULTI-TENANT DATA ISOLATION

The system organizes data into multi-tenant partitions where every operational record is scoped to a business tenant ID:

Business Scope: All products, batches, sales, and purchase orders carry a mandatory businessId attribute.

Index Enforcement: Compound indexes (businessId + sku, businessId + invoiceNumber) ensure high query performance.

Tenant Boundary: API middleware validates that authenticated users can only query and mutate their own business records.

A user from Tenant A cannot access, modify, or view records belonging to Tenant B, ensuring complete cloud privacy.

Automated integration tests verify that cross-tenant access attempts return 404 Not Found, confirming zero data leakage.

ShelfSense Platform Architecture

The proposed platform uses the following modular architecture:

Presentation Tier

React 18 single-page application with Vite, Tailwind CSS, and Recharts

API Routing & Middleware Tier

Express.js REST API with helmet security headers, CORS, and rate-limiting

Authentication & Authorization Tier

JWT access tokens, rotating refresh tokens, and role-based access control (RBAC)

FEFO Inventory Engine Tier

Earliest-expiry lot allocation, expired batch quarantine, and stock synchronization

Point of Sale (POS) Service Tier

Barcode lookup, cart calculation, tax addition, and receipt generation

Procurement & Receiving Tier

Purchase order lifecycle tracking with automated batch lot generation

Database Persistence Tier

MongoDB Atlas cloud cluster with 11 relational collections and compound indexes

EXPLANATION OF THE PLATFORM TIERS

1. Presentation Tier: Renders responsive dashboard interfaces, real-time POS checkout forms, and interactive analytics charts.

2. API Routing Tier: Dispatches incoming client requests to respective domain controllers while enforcing security headers.

3. Authentication Tier: Authenticates user credentials using bcrypt password comparisons and issues signed JWT tokens.

4. FEFO Engine Tier: Sorts active batches by expiry date and executes atomic deductions with concurrency protection.

5. POS Service Tier: Computes line-item totals, applies promotional discounts, calculates taxes, and creates digital invoices.

6. Procurement Tier: Tracks vendor orders and instantiates new product lots with manufacturing dates upon delivery receipt.

7. Database Tier: Stores persistent business records in MongoDB Atlas with compound indexes ensuring multi-tenant isolation.

INVENTORY LIFECYCLE MANAGEMENT

The inventory lifecycle spans procurement, batch creation, shelf monitoring, FEFO sales deduction, and movement auditing.

When goods arrive, the warehouse manager receives the purchase order, which triggers the automated creation of ProductBatch records.

Batches are tagged with manufacturing dates, expiry dates, purchase costs, and supplier references, updating the product totalStock.

Automated background routines evaluate batch shelf lives and classify them into proactive alert buckets to notify staff in advance.

When batches expire, they are automatically quarantined from sales checkout, eliminating the distribution of spoiled merchandise.

IMPLEMENTATION DETAILS

The project is implemented in TypeScript across frontend and backend, using Node.js, Express, React, and MongoDB Atlas.

The implementation is divided into the following modules:

Configuration and environment: Loads verified environment variables for ports, secrets, and database connection strings.

Database connection: Connects to MongoDB Atlas cluster with automated fallback to embedded memory server for testing.

Authentication and RBAC: Issues JWTs and enforces permissions for Business Owner, Inventory Manager, and Cashier roles.

Product and category management: Manages SKU creation, barcode indexing, pricing, and reorder threshold monitoring.

Batch and expiry tracking: Manages lots, calculates shelf lives, and categorizes batches into four temporal horizons.

FEFO allocation engine: Implements earliest-expiry deduction logic and concurrency-safe batch decrements.

POS checkout terminal: Provides barcode searching, cart management, tax calculations, and receipt generation.

Procurement and purchasing: Manages purchase order lifecycles and creates inventory batches upon goods receipt.

Inventory movement ledger: Records immutable logs of every stock addition, deduction, damage, and adjustment.

Business intelligence and reports: Aggregates real-time daily sales, monthly revenue, inventory valuation, and top SKUs.

Security audit logging: Captures operator identities, actions, target entity IDs, timestamps, and client IP addresses.

ALGORITHM

Input: Product ID, requested checkout quantity, cashier ID, and business tenant ID.

Output: Completed sales invoice with allocated batch lots and updated inventory balances.

Algorithm steps:

Start the checkout procedure.

Receive product ID and requested checkout quantity from POS scanner.

Validate that the product exists, is active, and belongs to the current business tenant.
Query active batches: SELECT * FROM ProductBatches WHERE productId == id AND expiryDate > now.https://archive.ics.uci.edu/dataset/967/phiusiil%2Bphishing%2Burl%2Bdataset
Filter out any batches where isExcludedFromSales == true (quarantined lots).
Sort eligible batches in ascending chronological order of expiration: ORDER BY expiryDate ASC.
Sum the available quantities across all eligible non-expired batches.
If total available non-expired stock < requested quantity, abort and return 400 Bad Request.
Initialize remainingQuantityToDeduct = requestedQuantity.
For each batch in sorted batches: calculate take = min(batch.availableQuantity, remainingQuantityToDeduct).
Decrement batch.availableQuantity by take and record batch allocation details.
Subtract take from remainingQuantityToDeduct. If remaining equals 0, break loop.
Commit batch decrements atomically to the database.
Recalculate product aggregate totalStock and update product record.
Create immutable record in InventoryMovement collection with movementType = 'SALE'.
Check if product totalStock <= minStockThreshold; if true, create low-stock notification.
Compute subtotal, discounts, applied sales taxes, and final grand total.
Generate unique invoice number and create Sale record in database.
Log transaction details to AuditLog collection with operator ID and timestamp.
Return completed sale record to client and display digital receipt.
Stop the procedure.

PERFORMANCE EVALUATION METRICS

The project uses several evaluation metrics to measure system correctness, operational reliability, and security integrity.

1. Authentication Success Rate: Measures correct validation of user credentials and proper issuance of JWT access and refresh tokens.
2. RBAC Enforcement Accuracy: Measures whether unauthorized role access attempts are strictly blocked with HTTP 403 Forbidden.
3. FEFO Chronological Precision: Measures whether sales deductions deplete the earliest-expiring batch before touching newer lots.
4. Stock Consistency Under Concurrency: Measures whether simultaneous checkouts maintain accurate batch balances without negative drift.
5. Multi-Tenant Data Isolation: Measures whether cross-tenant query attempts return 404 Not Found, confirming zero data leakage.
6. Movement Ledger Completeness: Verifies that every inventory modification produces an immutable before/after ledger entry.

Here, TP means verified operations, TN means properly rejected unauthorized requests, and FP/FN represent security or balance errors.

RESULTS AND DISCUSSION

The project evaluates the complete platform using an automated integration test suite executed with Vitest and Supertest against an active database.

The test suite systematically verifies tenant onboarding, authentication, role permissions, SKU creation, FEFO batch allocation, and data isolation.

The test results confirm that all 12 test assertions pass with 100% success rate, validating that the platform functions correctly under strict TypeScript.
The evaluation results demonstrate that the FEFO engine correctly prioritizes older batches and prevents checkouts when fresh stock is insufficient.
The actual results obtained from running the integration test suite are presented below:

PLATFORM INTEGRATION TEST RESULTS

Executed using Vitest and Supertest against an active MongoDB database instance.
Test Suite Pass Rate : 12 / 12 Tests Passed (100.00%)
Authentication Rate : 100.00% (Token issuance & refresh rotation verified)
RBAC Precision : 100.00% (Cashier restricted from SKU creation with 403)
FEFO Deduction Math : 100.00% (Earliest lot depleted first before newer lot)
Multi-Tenant Privacy : 100.00% (Tenant 2 isolated from Tenant 1 records with 404)
Integration test breakdown values
Tenant Registration : PASSED (Owner account & business created with JWTs)
Duplicate Prevention : PASSED (400 Bad Request enforced on duplicate email)
Over-Quantity Guard : PASSED (Checkout exceeding fresh stock rejected with 400)
Barcode SKU Query : PASSED (Instant product retrieval by barcode identifier)

APPLICATIONS

Supermarkets and Grocery Stores: Automates shelf rotation and reduces waste across dairy, baked goods, and fresh foods.
Retail Pharmacies and Chemists: Ensures patient safety by strictly preventing the sale of expired medications.
Convenience Stores and Bodegas: Provides fast barcode scanning, stock monitoring, and POS billing.
Wholesale Distributors and Warehouses: Tracks bulk pallet shipments, lot numbers, and supplier deliveries.
Food Service and Quick Service Restaurants: Manages perishable ingredients and enforces food safety standards.
FMCG Supply Chain Operations: Maintains full batch traceability for auditing, recalls, and supplier returns.
These are practical commercial applications. The platform provides a complete SaaS system ready for real-world store deployment.

ADVANTAGES

Automates First Expiry, First Out (FEFO) allocation directly during sales checkout.
Categorizes expiring stock into intuitive temporal horizons (<=7d, <=30d, <=60d, Expired).
Automatically quarantines expired stock, preventing accidental sales to customers.
Features strict end-to-end TypeScript checking, eliminating common runtime type errors.
Provides an immutable ledger documenting all stock additions, deductions, damages, and returns.
Streamlines supplier procurement with automatic lot generation upon purchase order receipt.
Enforces granular role-based permissions separating Admin, Manager, and Cashier responsibilities.

Ensures complete multi-tenant cloud data isolation across all database operations.

LIMITATIONS

Requires an active internet connection for real-time cloud database synchronization.

Physical shelf rotation relies on staff adhering to the system's batch deduction guidance.

High-speed barcode scanning requires compatible external USB or Bluetooth handheld scanners.

The current implementation manages single-store locations rather than multi-warehouse transfers.

Receipt printing depends on browser print dialogs unless dedicated thermal printer drivers are installed.

Initial system setup requires entering product catalog details and initial batch inventory.

Historical analytics require ongoing sales data to produce meaningful trend visualizations.

Store staff must be trained to record incoming deliveries with accurate batch and expiry dates.

FUTURE ENHANCEMENTS

The project can be extended in several impactful directions:

ESC/POS Thermal Printer Integration: Enable direct hardware printing of receipts via WebUSB.

Machine Learning Demand Forecasting: Predict future stock demand using historical sales trends.

Mobile Camera Barcode Scanning: Integrate camera-based barcode scanning using WebAssembly.

Multi-Warehouse Stock Transfers: Support inter-branch transfers between distributed retail locations.

Automated Vendor Reordering: Automatically generate purchase orders when stock reaches threshold limits.

Customer Loyalty and Credit Accounts: Support store credit, customer accounts, and promotional loyalty points.

Supplier Performance Analytics: Score vendors based on delivery timeliness and product defect rates.

Mobile Progressive Web App (PWA): Enable offline-capable mobile auditing and stock counting.

Automated Markdown Promotions: Automatically apply discounted pricing to batches nearing expiration.

Accounting Software Export: Integrate sales and purchase logs with standard accounting packages.

CONCLUSION

This project presents ShelfSense, an intelligent, multi-tenant inventory and expiry management platform built with the MERN stack and strict TypeScript. The system resolves the longstanding challenge of perishable stock spoilage by replacing generic numeric counters with date-aware batch management and automated First Expiry, First Out (FEFO) stock allocation.

The platform incorporates complete operational workflows, including barcode-enabled POS billing, vendor purchase order management, multi-tier expiry horizon tracking, immutable movement logging, and role-based access control. Comprehensive integration

testing demonstrated 100% test pass rates across authentication, RBAC enforcement, FEFO deduction accuracy, and multi-tenant data isolation.

The implementation evaluates the system using 12 integration tests verifying security, concurrency, and data isolation. It also provides an intuitive web interface for managing products, tracking batches, and analyzing store performance.

ShelfSense demonstrates how modern web architectures and domain-driven design can provide small and medium retailers with enterprise-level inventory intelligence, reducing food and medical waste while maximizing business profitability.

REFERENCES

Silver, E. A., Pyke, D. F., and Peterson, R., Inventory Management and Production Planning and Scheduling, 3rd ed., John Wiley and Sons, 1998.https://archive.ics.uci.edu/dataset/967/phiusiil+phishing+url+dataset

Nahmias, S., Perishable Inventory Systems, International Series in Operations Research and Management Science, Springer, 2011.https://www.tensorflow.org/

Chodorow, K., MongoDB: The Definitive Guide: Powerful and Scalable Data Storage, 2nd ed., O'Reilly Media, 2013.https://keras.io/

Banks, A., and Porcello, E., Learning React: Modern Patterns for Developing React Applications, 2nd ed., O'Reilly Media, 2020.https://scikit-learn.org/

Bierman, G., Abadi, M., and Torgersen, M., Understanding TypeScript, In European Conference on Object-Oriented Programming (ECOOP), Springer, 2014.https://docs.python.org/3/

Express.js Foundation, Express - Fast, unopinionated, minimalist web framework for Node.js. Available at: https://expressjs.com/